JAVA NOTES — OPERATORS & EXPRESSIONS

10-page study notes • Java operators, precedence and evaluation

1. What is an operator?

An operator is a symbol or keyword that performs an operation on one or more operands. An expression combines operands and operators to produce a value.

```
int total = price + 10;
//  + is operator; price and 10 are operands
```

Operator categories

- *Arithmetic: +, -, *, /, %*
- *Unary: +, -, ++, --, !, ~*
- *Relational / equality: <, >, <=, >=, ==, !=*
- *Logical / bitwise: &&, ||, &, |, ^*
- *Shift: <<, >>, >>>*
- *Conditional: ? :*
- *Type test: instanceof*
- *Assignment: =, +=, -=, *=, /=, %= and others*

Operands and result

```
int a = 8;
int b = 3;
int answer = a * b + 2; // 26
```

Remember: The meaning of an expression depends on its operators, operand types, precedence and evaluation rules.

2. ARITHMETIC & UNARY OPERATORS

Java language fundamentals • Handwritten-style revision notes

Arithmetic operators

Operator	Meaning	Example (a=8, b=3)
+	Addition	a + b = 11
-	Subtraction	a - b = 5
*	Multiplication	a * b = 24
/	Division	a / b = 2
%	Remainder	a % b = 2

Integer and floating-point division

```
System.out.println(7 / 2);    // 3
System.out.println(7 / 2.0);  // 3.5
System.out.println(7 % 2);    // 1
```

If both operands are integers, division produces an integer and truncates toward zero. The remainder operator % gives the remainder.

Unary operators

- *+x: unary plus; indicates a positive value.*
- *-x: unary minus; negates a numeric value.*
- *++x / x++: increment by one.*
- *--x / x--: decrement by one.*
- *!x: logical NOT for boolean values.*
- *~x: bitwise complement for integral values.*

```
int x = 5;
int a = ++x; // x=6, a=6 (pre-increment)
int b = x++; // b=6, then x=7 (post-increment)
```

Remember: Use ++ and -- carefully inside larger expressions; separate statements are often easier to read.

3. RELATIONAL & EQUALITY OPERATORS

Java language fundamentals • Handwritten-style revision notes

Relational operators

Relational operators compare numeric values and produce a boolean result.

```
int age = 20;
System.out.println(age > 18);  // true
System.out.println(age <= 18); // false
```

- *< less than; > greater than.*
- *<= less than or equal; >= greater than or equal.*

Equality operators

The equality operators compare two operands and return true or false.

```
int a = 10, b = 10;
System.out.println(a == b); // true
System.out.println(a != b); // false
```

Primitive vs reference equality

- *For primitive values, == compares the values.*
- *For object references, == checks whether both references refer to the same object.*
- *Use equals() when you want logical/content equality and the class defines it appropriately (for example, String).*

```
String first = new String("Java");
String second = new String("Java");

System.out.println(first == second);      // false
System.out.println(first.equals(second)); // true
```

Remember: Do not confuse = (assignment) with == (equality comparison).

4. LOGICAL & BITWISE OPERATORS

Java language fundamentals • Handwritten-style revision notes

Logical operators

Logical operators work with boolean expressions.

Operator	Meaning	Example
&&	Conditional AND	true && false → false
||	Conditional OR	true || false → true
!	Logical NOT	!true → false

Bitwise operators

Bitwise operators work on individual bits of integral values. &, | and ^ can also be used with boolean operands, where they evaluate both sides.

```
int a = 5; // 0101
int b = 3; // 0011

System.out.println(a & b); // 1  (0001)
System.out.println(a | b); // 7  (0111)
System.out.println(a ^ b); // 6  (0110)
System.out.println(~a);    // -6
```

Truth table (logical)

A	B	A && B	A || B
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

Remember: && and || short-circuit. Single & and | with booleans evaluate both operands.

5. SHIFT OPERATORS

Java language fundamentals • Handwritten-style revision notes

What do shift operators do?

Shift operators move the bits of an integral value left or right. They are useful in low-level programming, bit masks and binary operations.

Three shift operators

- *<< left shift: shifts bits left and fills the rightmost positions with zeros.*
- *>> signed right shift: shifts right and copies the sign bit on the left.*
- *>>> unsigned right shift: shifts right and fills left positions with zeros.*

```
int x = 8;        // ...00001000
System.out.println(x << 1);  // 16
System.out.println(x >> 1);  // 4
System.out.println(x >>> 1); // 4
```

Negative number example

```
int n = -8;
System.out.println(n >> 1);  // -4
System.out.println(n >>> 1); // large positive int
```

Shift distance

For int, only the lowest 5 bits of the right operand are used (distance 0–31). For long, only the lowest 6 bits are used (distance 0–63).

Remember: A left shift may discard high-order bits. Do not assume x << 1 is always safe multiplication by 2 for every int.

6. TERNARY / CONDITIONAL & instanceof

Java language fundamentals • Handwritten-style revision notes

Conditional (ternary) operator

The conditional operator uses three operands and selects one of two expressions based on a boolean condition.

```
condition ? valueIfTrue : valueIfFalse
```

```
int age = 20;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result); // Adult
```

- *The condition must be boolean.*
- *Only the selected branch is evaluated.*
- *The operator produces a value, so it can be used in assignments and return statements.*

instanceof operator

The `instanceof` *operator tests whether an object is an instance of a specified reference type (including compatible subclasses or implementations).*

```
Object value = "Hello";
if (value instanceof String) {
    System.out.println("It is a String");
}
```

Pattern matching instanceof

```
Object obj = "Java";
if (obj instanceof String text) {
    System.out.println(text.toUpperCase());
}
```

Pattern matching binds a variable when the test succeeds. This form is supported in modern Java versions.

Remember: For a null reference, instanceof returns false.

7. ASSIGNMENT & COMPOUND ASSIGNMENT

Java language fundamentals • Handwritten-style revision notes

Simple assignment

The = operator assigns the value of the right-hand expression to the left-hand variable.

```
int score;
score = 90;
String name = "Asha";
```

Compound assignment

Compound assignment combines an operation and assignment.

Operator	Meaning
+=	add then assign
-=	subtract then assign
*=	multiply then assign
/=	divide then assign
%=	remainder then assign
&=, |=, ^=	bitwise then assign
<<=, >>=, >>>=	shift then assign

```
int n = 10;
n += 5;  // 15
n *= 2;  // 30
```

Implicit conversion in compound assignment

```
byte b = 10;
// b = b + 1; // error: expression is int
b += 1;        // allowed; converted to byte
```

Remember: Compound assignment includes an implicit conversion to the left-hand type; it can still cause overflow or data loss.

8. PRECEDENCE & ASSOCIATIVITY

Java language fundamentals • Handwritten-style revision notes

Precedence

Precedence determines which operator is grouped first when an expression contains different operators.

```
int result = 2 + 3 * 4;
// multiplication first: 2 + 12 = 14

int grouped = (2 + 3) * 4; // 20
```

Common precedence order (high to low)

Level	Operators
1	postfix: expr++, expr--
2	unary: ++, --, +, -, !, ~, cast
3	multiplicative: *, /, %
4	additive: +, -
5	shift: <<, >>, >>>
6	relational: <, >, <=, >=, instanceof
7	equality: ==, !=
8	bitwise AND, XOR, OR: &, ^, |
9	logical AND, OR: &&, ||
10	conditional: ? :
11	assignment: =, +=, -=, ...

Associativity

Associativity determines grouping when operators have the same precedence.

- *Most binary operators group left to right:* `10 - 3 - 2` *means* `(10 - 3) - 2`.
- *Assignment and the conditional operator group right to left.*

```
int a, b, c;
a = b = c = 5; // a = (b = (c = 5))
```

Remember: Use parentheses when they make the intended grouping clearer, even if precedence already determines it.

9. SHORT-CIRCUIT & EVALUATION ORDER

Java language fundamentals • Handwritten-style revision notes

Short-circuit evaluation

With &&, Java skips the right operand if the left operand is false. With ||, it skips the right operand if the left operand is true.

```
int x = 0;
if (x != 0 && 10 / x > 1) {
    System.out.println("yes");
}
// safe: second test is skipped
```

```
boolean loggedIn = true;
boolean allowed = loggedIn || checkPassword();
// checkPassword() is skipped
```

Single & and | do not short-circuit

```
boolean a = false;
boolean b = a & methodCall(); // methodCall runs
boolean c = a && methodCall(); // methodCall skipped
```

Evaluation order

- *Java evaluates operands from left to right.*
- *Method arguments are evaluated left to right.*
- *The left operand of && and || is evaluated first; the right side may be skipped.*
- *For ?:, the condition is evaluated first and only one branch is evaluated.*
- *Operator precedence controls grouping; it does not mean operands are evaluated in a different left-to-right order.*

```
int n = 1;
int result = n++ + n++;
// first n++ gives 1, then 2nd gives 2
// result = 3; final n = 3
```

Remember: Avoid complicated expressions with multiple side effects. Split them into separate statements for clarity.

10. QUICK REVISION & PRACTICE

Java language fundamentals • Handwritten-style revision notes

Fast revision

- *Arithmetic: +, -, *, /, %. Integer division truncates toward zero.*
- *Unary: ++, --, +, -, !, ~.*
- *Relational: <, >, <=, >=. Equality: ==, !=.*
- *Logical: &&, ||, !. Bitwise: &, |, ^, ~.*
- *Shifts: <<, >>, >>>.*
- *Ternary: condition ? value1 : value2.*
- *instanceof checks an object's compatible reference type; null gives false.*
- *Assignment =; compound forms include +=, -=, *=, /=, %= and shift/bitwise forms.*
- *Precedence groups operators; associativity groups equal-precedence operators.*
- *&& and || short-circuit; Java evaluates operands left to right.*

Practice questions

- *1. What is 7 / 2 with int operands? Answer: 3.*
- *2. What does == compare for object references? Answer: reference identity.*
- *3. Which operator skips evaluation of the right side when left is false? Answer: &&.*
- *4. What is 2 + 3 * 4? Answer: 14.*
- *5. What is the result of 8 & 3? Answer: 0.*
- *6. What does instanceof return for null? Answer: false.*

Predict the output

```
int a = 5;
System.out.println(a++ + ++a); // ?
System.out.println(9 / 2.0);   // ?
System.out.println(4 << 1);    // ?
System.out.println(false && (1/0 > 0)); // ?
```

Answers: 12, 4.5, 8, false. The final expression does not divide by zero because && short-circuits.